Primer: GLADIUS (ERM-IRL)

econirl package documentation

1 Problem and Motivation

Structural estimators (NFXP, CCP) require enumerating the full state space and storing transition matrices of size $(|\mathcal{A}|, |\mathcal{S}|, |\mathcal{S}|)$. This is infeasible for continuous or high-dimensional states. IRL estimators (MCE-IRL, AIRL) avoid this by learning from demonstrations, but either require transitions (MCE-IRL) or lack interpretable structural parameters (AIRL).

Kang et al. (2025) introduce GLADIUS (ERM-IRL), which parameterizes the Q-function and expected continuation value $\zeta(s, a) = \mathbb{E}[V(s')|s, a]$ with neural networks, trains them via alternating SGD, and then projects the learned neural Q-values onto linear features to recover interpretable structural parameters. This combines the flexibility of neural IRL with the interpretability of structural estimation.

The key theoretical result is a Polyak-Łojasiewicz (PL) condition on the composite objective (NLL plus Bellman MSE) that guarantees global convergence at rate $O(T^{-1/4})$ under alternating gradient descent-ascent.

2 Model and Notation

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale (= 1 by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\|s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

GLADIUS adds the following objects to the common DDC notation.

Symbol	Name	Definition
$Q_{\phi_1}(s, a)$	Neural Q-network	MLP mapping $(s, a) \to \mathbb{R}$
$\zeta_{\phi_2}(s, a)$	Neural EV-network	Approximates $\mathbb{E}[V(s')\|s, a]$
$V(s)$	Soft value from Q	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\rho_{BE}(Q)$	Bellman error	$\mathbb{E}[(Q(s, a) - r(s, a) - \beta\, \mathbb{E}[V(s')\|s, a])^2]$
$\rho_{TD}(Q)$	TD error	$\rho_{BE}(Q) + \beta^2 \operatorname{Var}(V(s')\|s, a)$

3 Derivations

3.1 The ERM-IRL Objective

The estimator jointly maximizes the log-likelihood of observed actions and minimizes the Bellman consistency error. The objective is

$$\max_{\phi_1} \underbrace{\ell^p(\phi_1)}_{\text{NLL}} - \lambda \underbrace{\rho_{BE}(Q_{\phi_1})}_{\text{Bellman penalty}}, \tag{1}$$

where $\ell^p(\phi_1) = \frac{1}{N}\sum_i \log \pi_{\phi_1}(a_i|s_i)$ is the conditional log-likelihood under the softmax policy induced by Q_{ϕ_1}, and ρ_{BE} is the mean squared Bellman error.

3.2 Bi-Conjugate Decomposition

The mean squared TD error decomposes as

$$\rho_{TD}(Q) = \rho_{BE}(Q) + \beta^2\, \mathbb{E}_P\big[(V(s') - \mathbb{E}[V(s')|s,a])^2\big]. \tag{2}$$

The second term is the irreducible transition variance. In the original paper with observed rewards, the Bellman error ρ_{BE} anchors Q-value levels. Kang et al. (2025) introduce a dual variable $\zeta_{\phi_2}(s,a) \approx \mathbb{E}[V(s')|s,a]$ to separate this variance from the Bellman constraint, yielding the bi-conjugate formulation

$$\rho_{BE}(Q) = \max_{\zeta}\; 2\,\mathbb{E}\big[(Q - r - \beta\zeta)\,\zeta\big] - \mathbb{E}\big[\zeta^2\big] + \mathbb{E}\big[V(s')^2\big]. \tag{3}$$

The inner maximization over ζ recovers $\zeta^* = \mathbb{E}[V(s')|s,a]$, eliminating double-sampling bias.

3.3 Action-Difference Projection

After convergence, the implied reward is $\hat{r}(s,a) = Q_{\phi_1}(s,a) - \beta\,\zeta_{\phi_2}(s,a)$. However, Q-values are identified only up to a state-dependent constant $c(s)$ (Ng et al., 1999; Magnac and Thesmar, 2002). The constant cancels in action differences:

$$\Delta r(s) = \hat{r}(s,a) - \hat{r}(s,0) = \theta^\top\big(\phi(s,a) - \phi(s,0)\big). \tag{4}$$

Structural parameters are recovered via least squares: $\hat{\theta} = (\Delta\Phi^\top\Delta\Phi)^{-1}\Delta\Phi^\top\Delta\hat{r}$ where $\Delta\Phi$ stacks the feature differences across states and $\Delta\hat{r}$ stacks the reward differences.

3.4 Identification and the IRL Bias

Without observed rewards, Q is trained via NLL only (behavioral cloning with neural networks). The Bellman penalty is computed for monitoring but does not flow gradients through Q, because doing so causes Q-value explosion (from $[-1.5, 1.5]$ to $[4548, 6782]$) without reward anchoring. This means the implied reward $\hat{r} = Q - \beta\zeta$ inherits a state-dependent bias from the unidentified constant $c(s)$. On the Rust bus, where maintaining stays near s but replacing jumps to $s = 0$, the constant propagates asymmetrically through transitions. The replacement cost is recovered within 8% but the operating cost is overestimated by roughly 40%. This is a structural limitation, not a tuning problem. Use NFXP or CCP when transitions are known.

4 Estimation Algorithm

Algorithm 1: GLADIUS (ERM-IRL) (Kang et al., 2025)

Input: Panel $\{(s_i, a_i, s_i')\}_{i=1}^N$, features $\phi(s,a)$, discount β, scale σ

`// Step 1: Initialize networks`
Q_{ϕ_1}: $\mathrm{MLP}(s,a) \to \mathbb{R}$, output scaled by $1/(1-\beta)$
ζ_{ϕ_2}: $\mathrm{MLP}(s,a) \to \mathbb{R}$, same architecture

`// Step 2: Alternating SGD (Algorithm 1 of Kang et al.)`
for *epoch* $= 1, \ldots, T$ **do**
  for *each mini-batch* $\{(s_i, a_i, s_i')\}$ **do**
    `// Even batches: update` ζ `(freeze` Q`)`
    $V(s') = \sigma \log \sum_a \exp(Q_{\phi_1}(s', a)/\sigma)$ `// stop-gradient on` Q
    $\zeta_{\phi_2} \leftarrow \zeta_{\phi_2} - \alpha \nabla_{\phi_2} \frac{1}{|\text{batch}|} \sum_i \big(\zeta_{\phi_2}(s_i, a_i) - V(s_i')\big)^2$
    `// Odd batches: update` Q `(freeze` ζ`)`
    $\ell = -\frac{1}{|\text{batch}|} \sum_i \log \mathrm{softmax}(Q_{\phi_1}(s_i, \cdot)/\sigma)_{a_i}$ `// NLL`
    $Q_{\phi_1} \leftarrow Q_{\phi_1} - \alpha \nabla_{\phi_1} \ell$ `// no Bellman gradient in IRL`
  end
  if *no improvement for 50 epochs* **then break**
end

`// Step 3: Extract reward and project`
$\hat{r}(s,a) = Q_{\phi_1}(s,a) - \beta\, \zeta_{\phi_2}(s,a)$
$\Delta r(s) = \hat{r}(s,a) - \hat{r}(s,0), \quad \Delta\phi(s) = \phi(s,a) - \phi(s,0)$
$\hat{\theta} = (\Delta\Phi^\top \Delta\Phi)^{-1} \Delta\Phi^\top \Delta\hat{r}$ `// OLS projection`

Output: $\hat{\theta}$ (projected structural parameters), Q_{ϕ_1}, $\hat{r}$, projection R^2

5 Results

Table 2: Multi-component bus (2 components, 20 bins, 400 states, $\beta = 0.99$). NFXP needs the full $(2, 400, 400)$ transition tensor. GLADIUS learns from raw transitions without storing P.

Metric	NFXP	GLADIUS
$\hat{\theta}_{\text{rc}}$	3.0517	2.8699
$\hat{\theta}_{\text{oc}}$	0.029563	-1.615601
Log-likelihood	-9685.00	-9666.69
Time (s)	11.8	90.9
Needs P tensor	Yes	No

GLADIUS recovers the replacement cost within % of the true value via the action-difference projection. The operating cost is overestimated due to the state-dependent identification constant discussed in Section 3.4. The projection R^2 indicates how well the neural reward surface is explained by linear features. NFXP recovers both parameters within 5% but requires the full transition matrix and is limited to tabular state spaces. GLADIUS is the appropriate choice when the state space is too large for tabular methods or when the analyst wants to check whether linear features are sufficient via the R^2 diagnostic.

Code. `gladius_results.py` generates Table 2. Estimator: `src/econirl/estimation/gladius.py`.

```
.venv/bin/python papers/econirl_package/primers/gladius/gladius_results.py
```

References

Enoch H. Kang, Hema Yoganarasimhan, and Lalit Jain. An empirical risk minimization approach for offline inverse RL and dynamic discrete choice model. *arXiv:2502.14131*, 2025.

Thierry Magnac and David Thesmar. Identifying dynamic discrete decision processes. *Econometrica*, 70(2):801–816, 2002.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations. In *ICML*, pages 278–287, 1999.